

Git Source Control for Databricks

Last updated by | Falek Miah | Jul 12, 2019 at 4:36 PM GMT+1

Contents

- Databricks CLI Preparation
- GIT CLI Preparation
- GIT Source Control and Databricks Workspace
 - Introduction
 - Workflow
 - Guidelines and Steps
 - 1. Clone repositories (repo)
 - 2. Map repositories
 - 3. Create and Checkout feature branch
 - 4. Pull from feature branch to Local Repository
 - 5. Push/Load (import_dir) to Databricks Workspace
 - 6. Do Work...
 - 7. Pull (export_dir) from Databricks Workspace
 - 8. Sync, Stage & Commit (Push) changes
 - 9. Code Review the feature branch
 - 10. Create Pull Request
 - 11. Approve & Commit changes development branch
- Workspace Folders

Databricks CLI Preparation

1. Download and Install Python:
<https://www.python.org/downloads/> 
2. Once installed, make sure you have path environment variable set for Python scripts
3. Now run the following command:

```
pip install databricks-cli
```
4. To connect to Databricks Workspace using Databricks-CLI you'll require a personal Databricks Token
<https://docs.databricks.com/api/latest/authentication.html> 
5. Run the following command to configure connection to Databricks Workspace using the token:

```
databricks configure -token
```

You'll be prompted for the Databricks Host, which in our case is <https://northeurope.azuredatabricks.net/?o=5628103188513068> and your personal token

**Please note: that the configuring databricks creates a new file called `.databrickscfg` in your current working directory. This directory contains the Databricks workspace URL as well as your token.*

Ideally you will be running databricks CLI commands from different folders. To enable that, add a environment variable called `DATABRICKS_CONFIG_FILE` (include the filename in the path).

When the token expires, you'll be able to replace it with a new one in the `.databrickscfg` file (or run the configure command again).

You can now use databricks CLI with Databricks Workspace.

GIT CLI Preparation

1. Install git. You may want to use the following link:
<https://git-scm.com/download/win>
2. Run the following commands for some basic configurations

```
git config --global user.name "<Your name>"  
git config --global user.email "<Your email address>"
```

You can now use git CLI to manage repositories.

GIT Source Control and Databricks Workspace

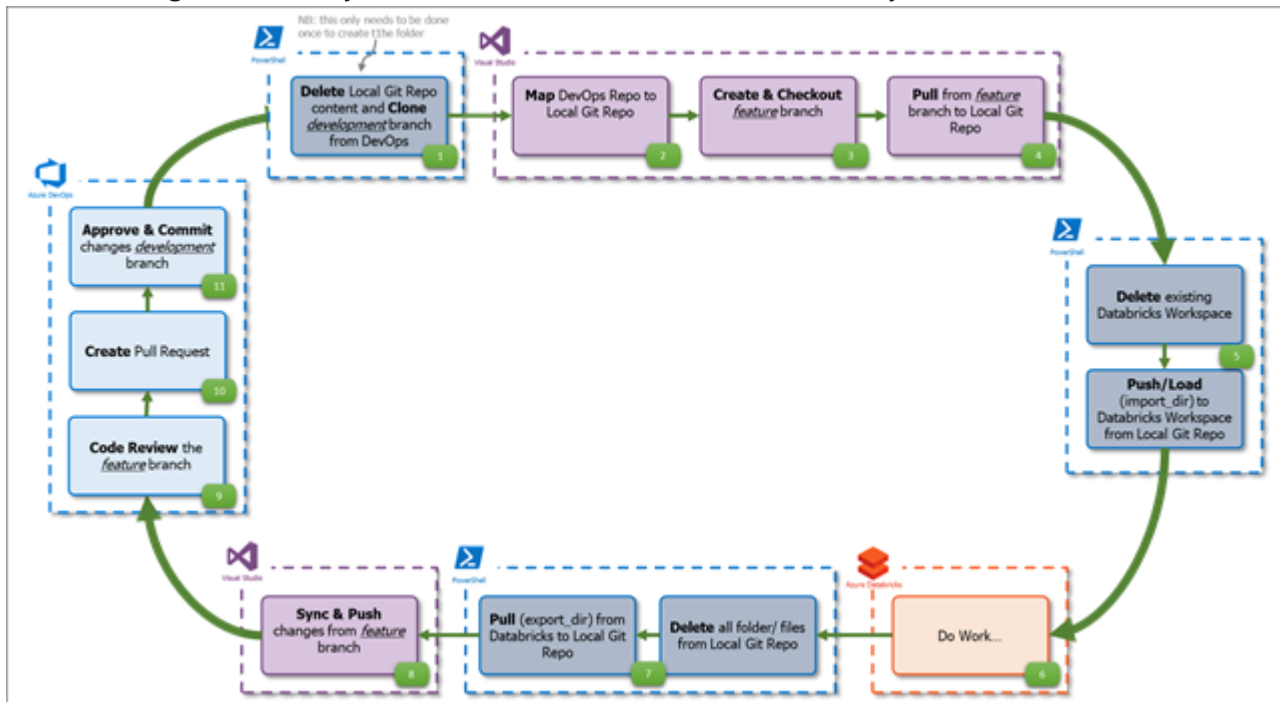
Introduction

Given that we have a shared Databricks Workspace and all developers are going to share it, we need a process for code management.

A mix between Visual Studios, PowerShell and Azure DevOps applications will allow you to Clone, Pull/Push and Publish code.

Workflow

The following workflow lays out the continuous source control lifecycle:



Git Source Control - Databricks.pptx

Guidelines and Steps

Please note: Download PowerShell scripts and Save in repo folder:

https://dev.azure.com/anglianwater/Data Services/_git/Data Services-Documentation?version=GBGitPowerShellScripts

1. Clone repositories (repo)

For all projects and tasks, the you must begin with cloning the default branch of the repository to your local machine before starting the activities.

The following steps will **Delete** any existing local Git repository content and **Clone** default branch from Azure DevOps to your local Git repository.

1. Open and run the following PowerShell scripts: `GitCmd_00_CloneDevOpsRepo.ps1`
2. Update the following variables:
 - Email (i.e. `userName@anglianwater.co.uk`)
 - Git local directory (i.e. `C:/Users/<userName>/source/reposDemo`) - destination path for storing the repository
3. If required, update the following variables:
 - Template path for PowerShell scripts (i.e. when different to Git local directory)
 - Databricks Folder & Workspace (i.e. AWBI)

It has been identified that the <https://dev.azure.com/anglianwater/> ... version of the URL doesn't work with git.

You can clone the desired repository using Visual Studio by connecting to DevOps Project and defining the destination path, as shown in figure 1.1.

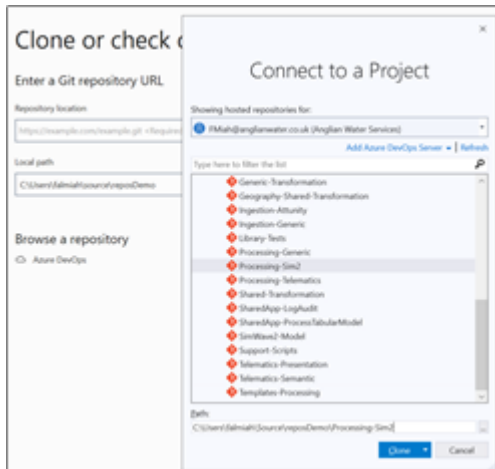


Figure 1.1 – Visual Studio Connect to a Project

2. Map repositories

You must ensure Visual Studio is connected to DevOps project and local Git repositories, as shown by the following steps and figure 2.1.

1. Open Visual Studio, under Team Explorer go to **Connect** section
2. Under **Local Git Repositories**, add the local repository destination path to DevOps

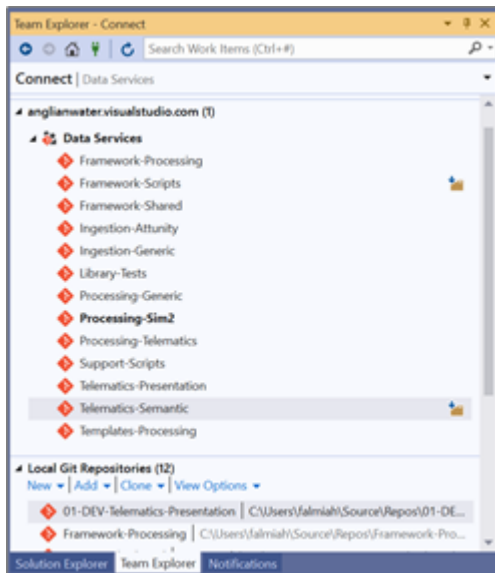


Figure 2.1 – Repositories mapped in Visual Studio

3. Create and Checkout feature branch

Every time a code change is required, a new *feature* branch must be created from the development branch.

You can create and checkout a *feature* branch from Visual Studio, as shown by the following steps and figure 2.1. Alternatively, you can create a *feature* branch in Azure DevOps portal and push to Visual Studio.

1. Open Visual Studio, under Team Explorer go to the **Branches** section
2. Under **New Branch**, enter Feature Name and click the Checkout Branch option

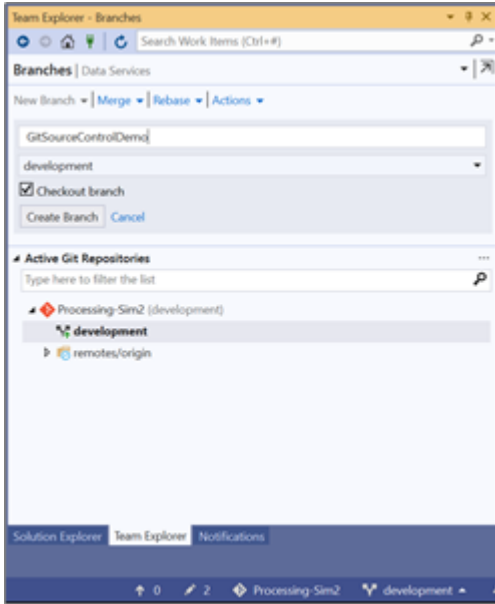


Figure 2.1 – Repositories mapped in Visual Studio

4. Pull from feature branch to Local Repository

After creating a *feature* branch in Visual Studio, you must synchronise the repositories, as shown by the following steps.

1. Open Visual Studio, under Team Explorer go to the **Branches** section
2. **Push** the *feature* branch to DevOps, as shown in figure 4.1.
 - If the *feature* branch is created through the Azure DevOps portal, then this step is not required.
3. **Fetch/Pull** from *feature* branch to ensure the branch is up to date, as shown in figure 4.2.

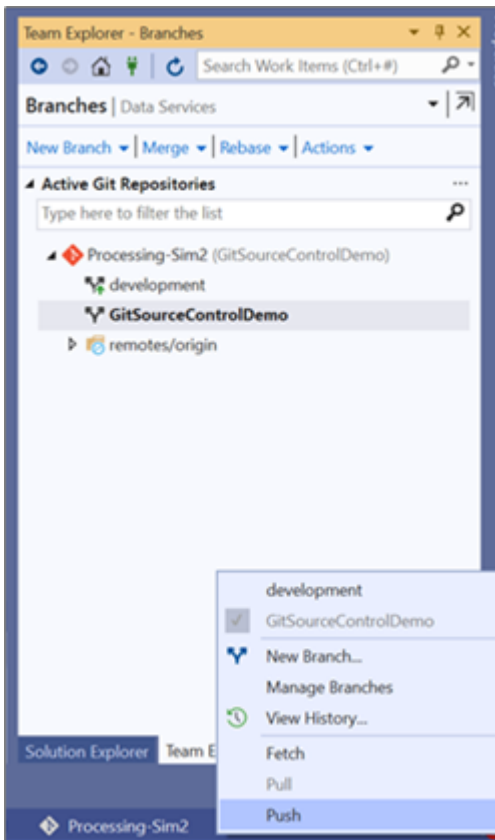


Figure 4.1- Push the *feature* branch

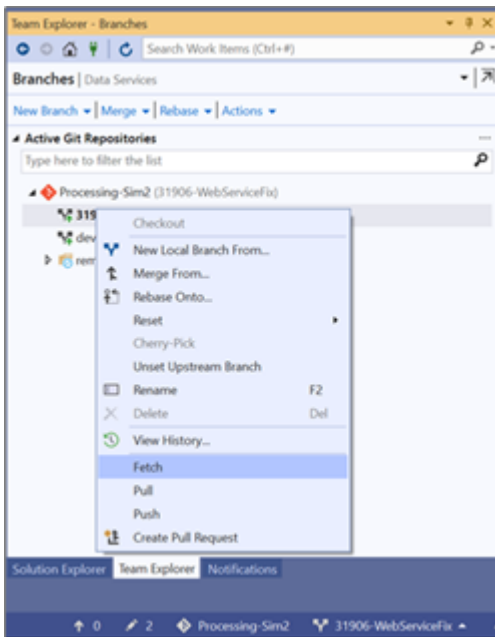


Figure 4.2- Fetch/Pull the *feature* branch

5. Push/Load (import_dir) to Databricks Workspace

You must deploy the repository content to your personal Databricks workspace area. A clean copy of the repository content must be import to the Databricks workspace, therefore before importing everything in your AWBI folder in the Databricks workspace must be deleted.

The following steps will **Delete** existing content from the Databricks workspace and **Push/Load** (import_dir) local Git repository content to Databricks workspace.

1. Open PowerShell scripts: **GitCmd_00_ExecuteGitDatabricksScripts.ps1**
2. Update the following variables:
 - Email (i.e. userName@anglianwater.co.uk)
 - Git local directory (i.e. C:/Users/<userName>/source/reposDemo) - destination path for storing the repository
3. If required, update the following variables:
 - Template path for PowerShell scripts (i.e. when different to Git local directory)
 - Databricks Folder & Workspace (i.e. AWBI)
4. Run the script and select **Push to Databricks**

6. Do Work...

You will make appropriate changes in the code in your Databricks workspace. After completing the changes, the following steps will help checking the code into the repository.

7.Pull (export_dir) from Databricks Workspace

The local Git repository content must be deleted before exporting the new content from your Databricks workspace.

The following steps will **Delete** existing repository content (folder/ files) from your local Git repository and **Pull** (export_dir) the new content from the Databricks workspace.

The delete only removes the defined repositories and files in the variable, therefore excludes `.git` folder and `readme.md` file.

1. Open PowerShell scripts: **GitCmd_00_ExecuteGitDatabricksScripts.ps1**
2. Update the following variables:
 - Email (i.e. userName@anglianwater.co.uk)
 - Git local directory (i.e. C:/Users/<userName>/source/reposDemo) - destination path for storing the repository
3. If required, update the following variables:
 - Template path for PowerShell scripts (i.e. when different to Git local directory)
 - Databricks Folder & Workspace (i.e. AWBI)
4. Run the script and select **Pull from Databricks**

8. Sync, Stage & Commit (Push) changes

You must synchronise, stage and commit (push) your changes from the feature branch to DevOps.

1. In Visual Studio, go to the **Sync** task under Team Explorer
2. **Stage** the changes
3. **Commit (Push)** the *feature* branch to DevOps, as shown in figure 8.1.

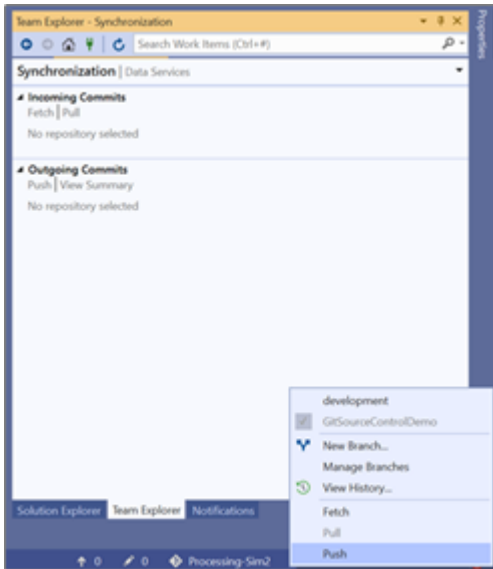


Figure 8.1- Sync, Stage and Push to DevOps

9. Code Review the feature branch

Only files have only been committed to your *feature* branch and not the *development* branch. You must do a code review in Azure DevOps before creating a Pull Request.

10. Create Pull Request

In Azure DevOps, create a **Pull Request** by following the on-screen instructions.

11. Approve & Commit changes development branch

In Azure DevOps, send to an appropriate Reviewer and once approved Commit/Complete the Pull Request which will merge the *feature* branch into the *development* branch.

Workspace Folders

